

```
# =====
# 1. BAYES THEOREM
# =====
```

```
print("1. BAYES THEOREM")
print("-" * 50)
```

```
1. BAYES THEOREM
-----
```

```
total_people = 100
di
sick_and_positive = 9

healthy_and_positive = 9
```

```
total_positive = sick_and_positive + healthy_and_positive
posterior_probability = sick_and_positive / total_positive
```

```
print("Total people:", total_people)
print("People with disease:", diseased_people)
print("Sick people who tested positive:", sick_and_positive)
print("Healthy people who tested positive:", healthy_and_positive)
print("Total positive results:", total_positive)

print("\nProbability of disease after positive test = {:.2f}%".format(
    posterior_probability * 100
))
```

```
Total people: 100
People with disease: 10
Sick people who tested positive: 9
Healthy people who tested positive: 9
Total positive results: 18

Probability of disease after positive test = 50.00%
```

```
print("2. CONCEPT LEARNING USING MAP")
print("-" * 50)
```

```
2. CONCEPT LEARNING USING MAP
-----
```

```
students = [
    [6, 90, 1],
    [6, 60, 0],
    [3, 90, 0],
    [5, 80, 1],
    [2, 40, 0]
]
```

```
def rule1(study_hours, attendance):
    if study_hours >= 5:
        return 1
    else:
        return 0
```

```
def rule2(study_hours, attendance):
    if attendance >= 75:
        return 1
    else:
        return 0
```

```
def rule3(study_hours, attendance):
    if study_hours >= 5 and attendance >= 75:
        return 1
    else:
        return 0
```

```
rules = {
    "Rule 1: Study Hours": rule1,
    "Rule 2: Attendance": rule2,
    "Rule 3: Study Hours AND Attendance": rule3
}
```

```
prior_probability = 1 / 3

best_rule = ""
highest_map_score = -1
```

```
for rule_name, rule_function in rules.items():

    correct_predictions = 0

    for study_hours, attendance, actual_result in students:

        predicted = rule_function(study_hours, attendance)

        if predicted == actual_result:
            correct_predictions += 1

    likelihood = correct_predictions / len(students)

    map_score = likelihood * prior_probability

    print("\n", rule_name)
    print("Correct Predictions:", correct_predictions)
    print("Likelihood:", round(likelihood, 2))
    print("Prior:", round(prior_probability, 2))
    print("MAP Score:", round(map_score, 4))

    if map_score > highest_map_score:
        highest_map_score = map_score
        best_rule = rule_name
```

```
Rule 1: Study Hours
Correct Predictions: 4
Likelihood: 0.8
Prior: 0.33
MAP Score: 0.2667
```

```
Rule 2: Attendance
Correct Predictions: 4
Likelihood: 0.8
Prior: 0.33
MAP Score: 0.2667
```

```
Rule 3: Study Hours AND Attendance
Correct Predictions: 5
Likelihood: 1.0
Prior: 0.33
MAP Score: 0.3333
```

```
print("\nBest Rule Selected:")
print(best_rule)
```

```
Best Rule Selected:
Rule 3: Study Hours AND Attendance
```

```
print("3. MAXIMUM LIKELIHOOD")
print("-" * 50)
```

```
3. MAXIMUM LIKELIHOOD
-----
```

```
actual_marks = [50, 60, 70]

model1_predictions = [49, 61, 69]

model2_predictions = [30, 80, 90]
```

```
def squared_error(actual, predicted):

    total_error = 0

    for actual_value, predicted_value in zip(actual, predicted):

        error = actual_value - predicted_value
        total_error += error ** 2

    return total_error
```

```
model1_error = squared_error(actual_marks, model1_predictions)
model2_error = squared_error(actual_marks, model2_predictions)

print("Actual Marks:", actual_marks)

print("\nModel 1 Predictions:", model1_predictions)
print("Model 1 Squared Error:", model1_error)

print("\nModel 2 Predictions:", model2_predictions)
print("Model 2 Squared Error:", model2_error)
```

```
Actual Marks: [50, 60, 70]

Model 1 Predictions: [49, 61, 69]
Model 1 Squared Error: 3

Model 2 Predictions: [30, 80, 90]
Model 2 Squared Error: 1200
```

```
if model1_error < model2_error:
    selected_model = "Model 1"
else:
    selected_model = "Model 2"

print("\nMaximum Likelihood selects:", selected_model)
```

```
Maximum Likelihood selects: Model 1
```